

Resenha Crítica: Artigo 4 — Design by Contract

Gustavo Pessoa Firmino Duarte

22 de Março de 2026

1 Introdução

Bertrand Meyer, criador da linguagem Eiffel, apresenta no artigo “Design by Contract” uma das contribuições mais elegantes à engenharia de software: a ideia de que a relação entre um módulo de software e seus clientes pode — e deve — ser formalizada como um contrato jurídico. Assim como em um contrato comercial existem obrigações e direitos para ambas as partes, na programação orientada a objetos cada método pode especificar exatamente o que exige de quem o chama (pré-condição), o que garante ao chamador após sua execução (pós-condição) e as propriedades que uma classe mantém consistentes em todos os momentos (invariante de classe). Essa abordagem, conhecida como *Design by Contract* (DbC), transforma a especificação de software de um exercício informal em uma ferramenta precisa e verificável.

2 Contratos, Pré-condições e Invariantes

O núcleo do DbC repousa em três elementos. A **pré-condição** define as responsabilidades do cliente: se o chamador não satisfaz a pré-condição, o comportamento do método é indefinido e a culpa é do chamador. A **pós-condição** define as responsabilidades do fornecedor: se a pré-condição foi atendida e o método retorna, a pós-condição precisa ser verdadeira. A **invariante de classe** especifica propriedades que devem valer após qualquer criação e após qualquer chamada de método público.

Uma consequência importantíssima dessa visão é a crítica à *programação defensiva*: Meyer argumenta que verificar a mesma condição tanto no fornecedor quanto no cliente é redundante e contrário ao espírito dos contratos. Se existe um contrato claro, o fornecedor não precisa se proteger de chamadas inválidas — essas chamadas simplesmente constituem um erro do cliente. Isso resulta em código mais limpo, sem camadas de validação duplicadas.

Meyer também aborda o tratamento de exceções através dos mecanismos *rescue* e *retry* do Eiffel: uma exceção é vista como uma violação contratual, e a cláusula *rescue* tem a responsabilidade de tentar restaurar o invariante antes de uma nova tentativa ou de propagar a falha. Por fim, o princípio de *subcontração* pela herança — subclasses podem apenas afrouxar pré-condições e fortalecer pós-condições, respeitando o Princípio de Substituição de Liskov — integra DbC naturalmente à hierarquia de herança.

3 Conexão com a Prática Profissional

Minha experiência como estagiário de Sistemas na ArcelorMittal, atuando no suporte a aplicações internacionais, evidenciou a importância de contratos claros entre módulos. Em vários sistemas legados que analisei, a ausência de pré-condições explícitas gerava um padrão típico: cada serviço validava entradas que já deveriam ter sido validadas pelo chamador, resultando em pilhas de `if (value != null && value.isValid())` espalhadas por toda a base de código. Esse é exatamente o antipadrão que Meyer critica ao defender o DbC contra a programação defensiva.

No meu MVP de e-commerce com Node.js e Prisma ORM, adotei uma prática parcialmente inspirada no DbC ao definir esquemas de validação na camada de API (usando Zod) e tratar esses esquemas como pré-condições: se a requisição não passa na validação, a camada de serviço nunca é acionada. A camada de serviço, por sua vez, assume que os dados estão válidos — exatamente a divisão de responsabilidades que Meyer propugna. Essa estratégia reduziu significativamente a quantidade de `try/catch` e validações redundantes no código de negócio.

Reconheço, porém, que invariantes de classe são quase inexistentes em JavaScript/TypeScript por padrão, e ferramentas como TypeScript apenas oferecem garantias estáticas parciais. Linguagens como Eiffel ou Kotlin com contratos explícitos representam uma visão ainda não amplamente adotada no ecossistema que utilizo.

4 Conclusão

O DbC de Meyer oferece uma mudança de paradigma: em vez de escrever código defensivo que tenta se proteger de todos os abusos possíveis, o desenvolvedor especifica com precisão o que é esperado e o que é garantido. Essa clareza beneficia a documentação, os testes e a manutenção. Embora poucos ambientes modernos suportem DbC nativamente como o Eiffel, os princípios de pré e pós-condições influenciam fortemente bibliotecas de validação, frameworks de testes por contrato e até especificações OpenAPI. A essência do contrato — obrigações e benefícios mútuos claramente definidos — é uma das ideias mais duradouras da engenharia de software.